Dawn Andrei Pamesa
TS31
Group members: Clarence Canaria, David Decano, Dawn Andrei Pamesa, Joboy Porras Jr., and Erold Jan P. Leocario

CS0011 Final Project

Part 1: Application Proposal

1. Application Title and Category

Title: MediMind: A Medication Reminder and Intake Tracker

Category: Health and personal care. Database-driven Android application.

2. Problem Statement and Target Users

People who take medicines every day, such as maintenance medicines for high blood pressure or diabetes, often forget a dose or cannot remember if they already took it. Paper lists and phone alarms do not record what was actually taken, so no one can see which doses were missed.

Target users

- Adults and older adults who take daily medicines.
- Family members who help manage a relative's medicines.

3. Proposed Solution

MediMind lets a user save each medicine with its dose and daily schedule. At each scheduled time the app shows a notification. The user taps Taken or Skip, and the app keeps a history. A simple report shows how many doses were taken and missed over the last 7 or 30 days. All records stay on the phone in a SQLite database and remain after the app is closed.

The app is a reminder and record keeper. It does not give medical advice.

4. Objectives

General objective. To develop an Android application that helps users take their medicines on time and keep a reliable record of their intake.

Specific objectives

1. To let users add, view, update and delete medicines and their daily schedules in a SQLite database.
2. To remind users at each scheduled time with a notification.
3. To record each dose as taken or skipped and show doses that were missed.
4. To show an adherence report for the last 7 and 30 days.
5. To provide clear validation, messages and empty states so the app is easy to use.

5. Core Features

- Medicine list with search and a filter for active and finished medicines.
- Add or edit a medicine: name, dose, instructions, start date, optional end date, one or more daily times, and days of the week.
- Medicine detail screen with the schedule and intake history.
- Today screen that lists today's doses with Taken and Skip buttons.
- Delete with a confirmation dialog.
- Messages for success, errors, empty lists and confirmations.

6. Database Technology and Data to Be Stored

Technology: SQLite through SQLiteOpenHelper, in Kotlin. Data stays on the device.

Data stored: medicines, their schedules, and a log of what the user did for each dose. There is no account and no online storage.

7. Tables, Fields, Primary Keys, and Relationships

medications

Field	Type	Notes
id	INTEGER	Primary key, autoincrement
name	TEXT	Required
dose	TEXT	Required, for example "500 mg"
instructions	TEXT	Optional, for example "after meals"
start_date	TEXT	yyyy-MM-dd, required
end_date	TEXT	Optional
is_active	INTEGER	1 or 0

schedules

Field	Type	Notes
id	INTEGER	Primary key, autoincrement
medication_id	INTEGER	Foreign key to medications.id, delete cascades
time_of_day	TEXT	HH:mm, required
days_of_week	TEXT	For example "1,2,3,4,5,6,7"

intake_logs

Field	Type	Notes
id	INTEGER	Primary key, autoincrement
schedule_id	INTEGER	Foreign key to schedules.id, delete cascades
dose_date	TEXT	yyyy-MM-dd
status	TEXT	TAKEN or SKIPPED
logged_at	TEXT	Date and time the user tapped

Relationships. One medication has many schedules. One schedule has many intake logs. A log is unique for each schedule and date.

Rule for missed doses. A dose is not stored until the user acts on it. A dose whose time has passed and has no log is counted as missed.

8. Screen and Navigation Plan

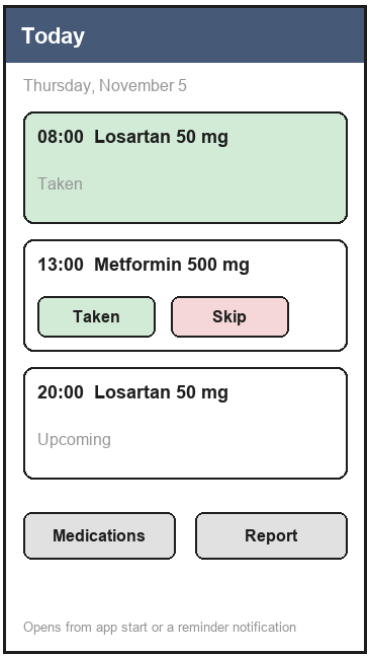
Screen	Purpose	Opens from
Today	Today's doses, Taken and Skip, empty state	App start, tapping a notification
Medications	List with search and filter, add button	Today (menu)
Medication Form	Add or edit a medicine and its times	Medications, Detail
Medication Detail	Schedule, history, edit and delete	Medications
Report	Taken and missed over 7 or 30 days	Today (menu)

Explicit Intents and data transfer

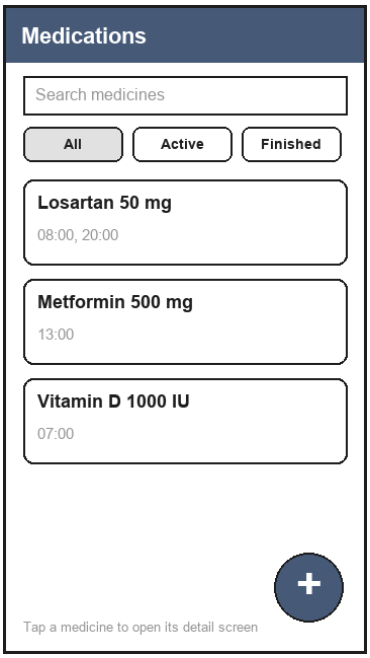
- Medications to Detail: explicit Intent with medication_id.
- Detail to Form (edit): explicit Intent with medication_id. Add opens the form with no id.
- Notification to Today: explicit Intent with schedule_id and the dose date.
- Form returns to Medications, which reloads the list.

9. Initial Wireframes

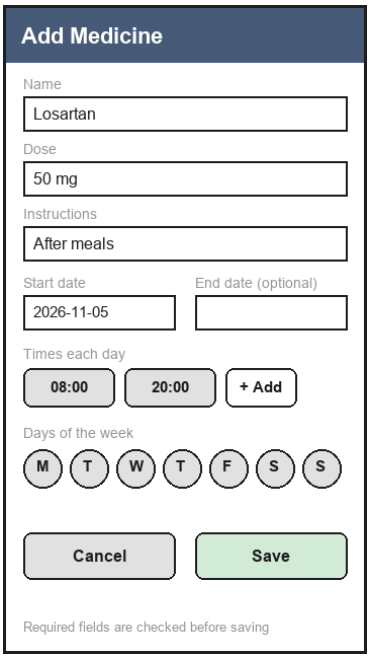
These are initial sketches of the five main screens. They use sample data and will be refined during development.



Today



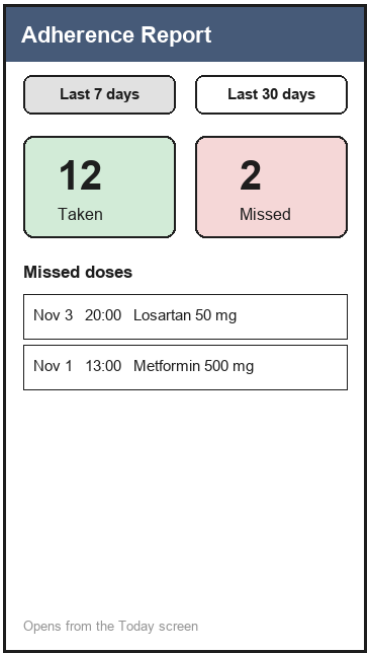
Medications



Medication Form



Medication Detail



Report

10. Additional Features Beyond CRUD

1. Dose reminders by notification at each scheduled time. This is the main extra feature.
2. Adherence report for the last 7 and 30 days.
3. Search and filter on the medicine list.

If time is short, the notification reminder and the adherence report are the two to protect. Search and filter can be simplified.

11. Development Schedule

Coding starts only after the instructor approves this proposal. The final due date is around the end of November, to be confirmed with the instructor. Testing is done as each screen is finished, so the 12 test cases are not left to the last week.

Phase	Dates	Work
1	Nov 5 to Nov 11	Submit this proposal, get instructor approval, set up the Android project and the group repository.
2	Nov 12 to Nov 18	Database class and models, Medication Form, Medications list, validation and messages.
3	Nov 19 to Nov 25	Detail screen, delete with confirmation, Today screen with Taken and Skip, notification reminders.
4	Nov 26 to Nov 30	Report, search and filter, testing (12 test cases, defects and fixes), improvements from testing, documentation, user guide, APK and final checks.

Task owners will be assigned by the group leader once the proposal is approved.

12. AI-Use Plan

This is an AI-assisted activity. The group will use AI assistants (for example Claude, Codex or Gemini) as a helper, not as a replacement for the group's own work.

What AI may be used for

- Explaining Kotlin, SQLite and Android Studio errors, and suggesting how to fix them.
- Suggesting test cases and checking that the proposal covers every requirement in the activity sheet.
- Reviewing code the group has written, and improving the wording of documentation.

What AI will not be used for

- Submitting AI output that the group has not read, run and understood.
- Entering real personal or health information into an AI tool.

How the output is checked

- Every piece of AI-suggested code is run on the emulator or a real phone before it is kept.

- The results are recorded in the 12 test cases, and any defect found is documented with its correction.
- Every member must be able to explain the code and the design they submit.

How the use is disclosed

- The documentation will include an AI-use log with the tool used, the task, what was accepted or changed, and how it was checked.

Risks and Limits

- Notifications on recent Android versions need the notification permission and, for exact times, special alarm handling. This will be tested early on a real phone or the emulator.
- Reminders may need to be set again after the phone restarts. The group will decide whether this is in scope or documented as a known limit.
- The app stores health information on the device only. It is a reminder tool and not medical advice, and the app says so.